ALGORITHM DESIGN PROBLEM SET

Topic: Advanced Insertion Sort & Selection Sort Variants • Progression Series

Question 1: Multi-Key Sorting with Stability Check

Difficulty: Base (100%)

You are given a list of employee records, each containing a Name, Department, and Base Salary. Your task is to sort these records using the **Selection Sort** algorithm. The primary sorting criterion is Base Salary in descending order. If two employees share the same salary, break the tie by sorting them lexicographically by Department in ascending order. If both fields match, sort them lexicographically by Name in ascending order.

> **Input Format:** First line contains integer N ($1 \le N \le 1000$). Next N lines contain comma-separated strings: "Name, Department, Salary".
> **Output Format:** Print sorted records, one per line, formatted as "Name, Department, Salary".

Sample Input	Sample Output

`4`
`Alice, Engineering, 80000`
`Bob, Marketing, 60000`
`Charlie, Engineering, 80000`
`David, HR, 50000`
	`Alice, Engineering, 80000`
	`Charlie, Engineering, 80000`
	`Bob, Marketing, 60000`
	`David, HR, 50000`

Question 2: K-Sorted Dynamic Window Shift

Difficulty: Medium (+25%)

An array of elements is defined as being K-sorted if each element is at most K positions away from its target sorted position. Implement an optimized version of the **Insertion Sort** algorithm that sorts a given array knowing this property beforehand. Your algorithm must only examine the sliding look-ahead window of size $K+1$ instead of scanning backward through the whole sorted array portion, improving structural performance.

> **Input Format:** The first line contains integers N and K ($1 \le K < N \le 10^5$). The second line contains N space-separated integers.
> **Output Format:** Print the fully sorted array separated by spaces.

Sample Input	Sample Output
`5 2` `3 2 1 5 4`	`1 2 3 4 5`

Question 3: Bi-Directional Dual-Pivot Selection

Difficulty: Advanced (+50%)

Standard selection sort finds either the minimum or maximum element and places it into position. Design a variant known as **Double Selection Sort**. In a single linear scan from both ends simultaneously, find *both* the absolute minimum and the absolute maximum elements within the unsorted boundaries. Swap the minimum to the beginning of the subarray and the maximum to the end of the subarray, narrowing down your search space from both boundaries at each step. Be extremely careful when managing elements that swap out of their positions prematurely.

> **Input Format:** The first line contains N ($1 \le N \le 2000$). The second line contains N integers.
> **Output Format:** Output the array after each external iteration of the dual-pivot loops to visualize boundaries collapsing.

Sample Input	Sample Output
`5` `40 10 30 50 20`	`10 40 30 20 50` `10 20 30 40 50`

Question 4: Online Stream Inversion Counting

Difficulty: Challenging (+75%)

You are building a real-time data ingestion pipe. As values streaming arrive one by one, insert them into a dynamically growing list while keeping the container organized using online **Insertion Sort**. For every element read, output the number of shifting shift operations (or element inversions) that had to take place before the element settled into its correct index.

> **Input Format:** First line contains an integer N ($1 \le N \le 5000$). Second line contains N integers representing the streaming live values.
> **Output Format:** Output N space-separated integers, where the i-th value indicates the shifts required for the i-th element.

Sample Input	Sample Output
`5` `2 1 5 4 3`	`0 1 0 1 2`

Question 5: Cost-Optimized Matrix Wave Alignment

Difficulty: Expert (+100%)

You are given an $M imes N$ matrix. You need to reorganize its elements so that each row alternates between ascending and descending sequences (Row 0 ascending, Row 1 descending, etc.). However, you cannot just sort normally. You must modify **Selection Sort** so that it calculates the total weight distance matrix of swapping indices: $Cost = |Row_{from} - Row_{to}| + |Col_{from} - Col_{to}|$. Minimize overall internal element relocation cost during selection passes.

> **Input Format:** First line contains row dimension M and column dimension N ($2 \le M, N \le 50$). The following M lines contain N space-separated integers.
> **Output Format:** Print the final customized path-aligned transformation matrix, and the absolute minimal aggregate cost index sum.

Sample Input	Sample Output

`2 3`
`7 3 5`
`1 9 4`
	`3 5 7`
	`9 4 1`
	`Total Minimized Cost: 6`